南京大学 计算机科学与技术系

软件工程课程实验报告

二手交易系统设计与建模
——实验二：软件设计与建模

课程名称	软件工程
实验名称	实验二：软件设计与建模
专　　业	计算机科学与技术
学　　号	251220154
姓　　名	王鸣骏
完成日期	2026 年 10 月 24 日

目 录

一、项目概述与软件主要功能 2

1.1 项目说明 2

1.2 软件的主要功能 2

1.3 功能模块划分 3

二、软件架构与 UML 设计 3

2.1 总体架构与数据存储设计 3

2.2 结构图（一）：类图 5

2.3 结构图（二）：组件图 5

2.4 行为图（一）：用例图 6

2.5 行为图（二）：状态图 7

2.6 行为图（三）：活动图 8

2.7 交互图：时序图 10

三、各功能模块 UI 界面设计与功能描述 11

3.1 设计规范与制作方式 11

3.2 界面 1：用户注册与登录（模块 M1） 11

3.3 界面 2：商品浏览与组合查询（模块 M2） 12

3.4 界面 3：商品详情（模块 M2/M3/M4/M5） 13

3.5 界面 4：商品发布与我的商品（模块 M2） 14

3.6 界面 5：我的预约与交易确认（模块 M3/M7） 15

3.7 界面 6：卖家预约处理与交付确认（模块 M3） 16

3.8 界面 7：个人中心与收藏（模块 M4/M7） 17

3.9 界面 8：管理员工作台（模块 M5/M6） 18

四、大模型辅助工具的使用记录与分析 19

4.7 优缺点综合分析 22

4.8 总结：大模型如何辅助软件设计，以及「人只做检查校对」的工作流 22

五、实验总结 23

附录：提交物清单 23

一、项目概述与软件主要功能

1.1 项目说明

本实验选择的项目为二手交易系统。系统面向买家、卖家与平台管理员三类用户，参考常见二手交易平台的核心流程，建立一个以线下交付为基础、以商品状态与预约记录为主线、范围可控可实现的校园二手交易平台。本报告承接实验一《二手交易系统需求分析报告》（共 26 条已确认需求：功能需求 19 条、非功能需求 4 条、约束 3 条），在其基础上完成软件架构设计、UML 建模与各功能模块的界面设计。

按照实验一确定的系统边界，第一版仅实现线下二手交易的核心闭环：商品发布、审核、查询浏览、预约、线下交付确认、收藏、举报与平台管理；明确不实现在线支付、平台担保、即时聊天、物流轨迹、信用评价、退款仲裁、关注与动态流等功能（REQ-021）。

1.2 软件的主要功能

依据实验一需求规格说明，软件的主要功能按功能域归纳如表 1-1 所示，每项功能均标注对应的需求编号，保证设计与需求可双向追溯。

功能域	主要功能点	对应需求
账户与权限	注册（手机号唯一、密码仅保存不可逆摘要）、登录、买家/卖家/管理员三类角色权限控制、未登录与越权访问拦截	REQ-001/002/018
商品管理	商品草稿创建与维护（标题/描述/分类/成色/价格/图片/取货方式七项必填）、提交审核、发布后修改、下架、管理员违规下架	REQ-002/003/005/011
商品审核	管理员审核待审核商品：通过后商品上架在售，驳回后退回草稿并记录原因；待审核商品被编辑后须重新审核	REQ-003/004
商品浏览	关键词/分类/价格区间/商品状态/发布时间多条件组合查询；商品详情仅展示卖家公开昵称；下架与违规下架商品不可见	REQ-007/022
预约与交易	买家预约在售商品（首个成功请求为准、唯一有效预约）；卖家确认或拒绝、买家取消；线下交付后卖家确认交付、买家确认收货，双方确认即交易完成	REQ-008/009/010/024
一致性与并发	商品七状态与预约五状态的转换约束；商品版本号乐观锁，旧页面与并发请求不得覆盖最新结果；关键写入原子事务	REQ-006/012/013/019
收藏与个人中心	收藏/取消收藏（同一商品唯一记录、状态同步）；个人中心提供我的商品与我的预约清单，不含社交功能	REQ-023/025
举报与处置	登录用户提交举报（同一用户同一商品至多一条待处理）；管理员驳回举报或确认违规并违规下架	REQ-026
平台管理	关键操作日志记录与四条件组合查询（成功与失败均记录）；按商品状态与分类的统计汇总	REQ-014/015/016
质量属性	商品查询性能（100 并发下 95% 请求 2 秒内返回）；密码安全与越权防护；主要页面 360px 以上视口可用	REQ-017/018/020

表 1-1 软件主要功能清单

1.3 功能模块划分

按照业务能力垂直划分业务模块、横切关注独立成支撑组件的原则，系统划分为八个模块，各模块职责与需求映射如表 1-2 所示。

编号	模块名称	职责	对应需求	对应界面
M1	认证与权限模块	注册、登录、会话令牌、角色权限校验与未登录拦截	REQ-001/002/018	界面 1
M2	商品模块	草稿维护、提交审核、修改下架、多条件查询、详情与可见性控制、金额与重复提交校验	REQ-002/003/005/007/011/022	界面 2/3/4
M3	预约与交易模块	并发预约、卖家确认/拒绝、买家取消、交付与收货双确认、版本号校验	REQ-008/009/010/012/024	界面 3/5/6
M4	收藏模块	收藏与取消收藏、收藏列表与商品状态同步	REQ-023	界面 7
M5	举报模块	举报提交与约束校验、举报记录维护	REQ-026	界面 8
M6	审核与管理模块	商品审核、违规下架、举报处置、日志查询、商品统计	REQ-004/014/015/016/026	界面 8
M7	个人中心模块	我的商品、我的预约、我的收藏的本人数据清单与状态筛选	REQ-025	界面 7
M8	公共支撑组件	操作日志组件、版本并发控制组件、参数校验组件（横切关注，被 M1-M7 复用）	REQ-012/014/019	—

表 1-2 功能模块划分与需求映射

模块划分的理由如下：其一，按业务能力（领域）垂直切分，每个模块对应一组高内聚的用例，模块间仅通过服务接口交互，耦合度低，便于分工实现与单元测试；其二，预约与交易从商品模块中独立出来，因为它是全系统并发与一致性要求最高的部分（唯一有效预约、乐观锁、双确认原子更新），独立成模块有利于集中处理技术风险；其三，日志、并发控制、参数校验属于横切关注，若分散在各业务模块中会造成大量重复代码，故抽象为公共支撑组件；其四，实验一的基本要求模块（用户管理、商品管理、审核、查询、预约交易、日志、统计、收藏、举报）全部保留，无一遗漏。

二、软件架构与 UML 设计

2.1 总体架构与数据存储设计

系统采用「客户端层 – 业务服务层 – 数据访问层」三层架构，配合横切的公共支撑组件：客户端层为响应式 Web 页面（360px 以上视口可用，满足 REQ-020），经 REST 接口调用业务服务层；业务服务层的六个业务模块（M1-M6）编排用例逻辑；数据访问层以仓储（Repository）模式封装持久化；操作日志组件以拦截器方式记录全部关键操作，版本并发控制组件以「状态 + 版本号」原子条件更

新实现首个成功请求为准的预约规则。模块间依赖单向向下，越权访问在认证与权限模块统一拦截。

数据采用关系数据库存储，共七张表，关键约束直接支撑一致性需求（表 2-1）。

数据表	关键字段	关键约束
用户表	用户Id、手机号、用户名、密码摘要、角色	手机号唯一索引；密码仅存不可逆摘要；用户名可重复
商品表	商品Id、卖家Id、标题、描述、分类、成色、价格、图片、取货方式、状态、版本号、发布时间	价格 CHECK (0, 1000000]；状态取值七种；状态变更与关键字段修改后版本号 +1；条件更新 WHERE 状态=期望状态 AND 版本=期望版本
预约表	预约Id、买家Id、商品Id、状态、创建时间	部分唯一索引保证同一商品至多一条有效预约（状态属于待确认或已确认）；已取消/已拒绝/已完成记录保留
交易确认表	确认Id、预约Id、卖家确认交付时间、买家确认收货时间	关联唯一有效预约；双方时间齐全方可触发交易完成
收藏表	收藏Id、买家Id、商品Id、创建时间	UNIQUE(买家Id, 商品Id)；商品售出或下架后记录保留并同步状态
举报表	举报Id、举报人Id、商品Id、原因、状态、处理人、处理时间、处理结果	同一用户对同一商品至多一条待处理举报
操作日志表	日志Id、操作者Id、操作类型、对象类型、对象Id、结果、原因、时间	按操作者/对象/类型/时间建组合索引，支撑四条件查询

表 2-1 数据存储设计与关键约束

选择关系数据库的原因：唯一性约束（手机号、有效预约、收藏、待处理举报）与事务原子性（REQ-013/019）是本系统的核心质量要求，关系库可由数据库层直接保证，避免应用层重复实现；版本号列以最小代价实现乐观锁（REQ-012），无需引入额外的分布式组件。

按照实验要求，本报告绘制了六种 UML 图，覆盖结构性、行为性、交互性三类，选用说明见表 2-2。

类别	选用图	选用理由
结构性图（静态图）	类图（必画）、组件图	类图给出领域模型与服务分层，是实现的直接蓝本；组件图表达三层架构与模块间依赖及横切组件
行为性图（动态图）	用例图、状态图、活动图	用例图对齐需求边界与角色权限；本系统核心是状态流转，状态图直接刻画商品与预约两个状态机；活动图以泳道呈现跨角色协作的全流程
交互性图	时序图	并发预约与版本冲突是本系统技术风险最高的场景，时序图最能表达消息时序与事务边界

表 2-2 三类 UML 图的选用说明

2.2 结构图（一）：类图

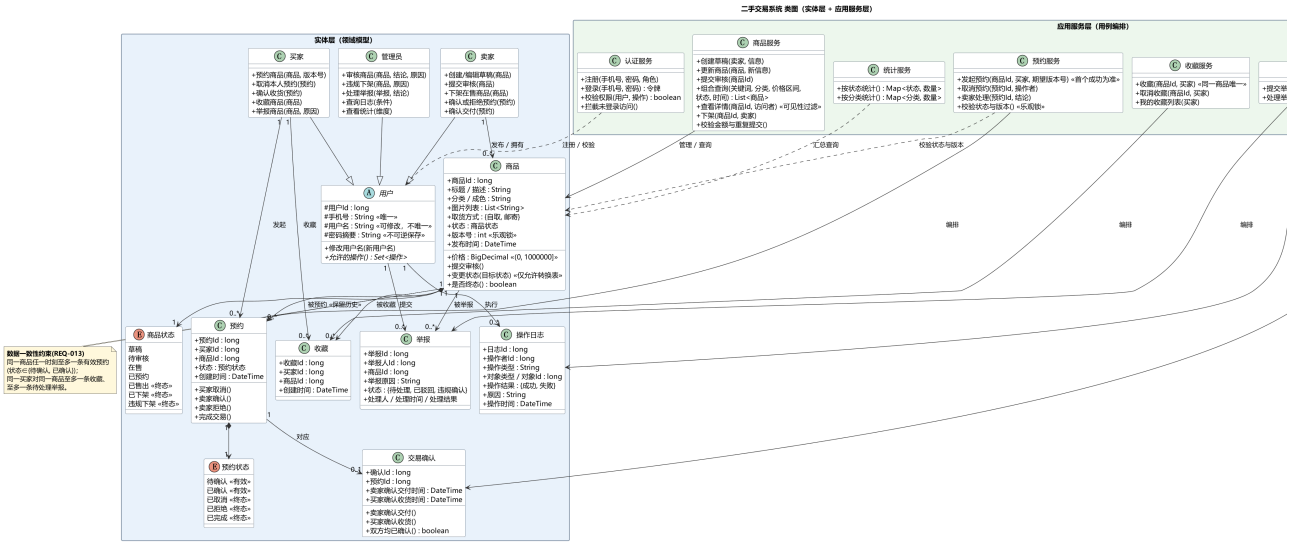


图 2-1 UML 类图（结构性图，必画）

类图分为实体层与应用服务层两层。实体层是领域模型：用户抽象父类保存公共属性（手机号唯一、密码不可逆摘要），买家、卖家、管理员三个子类以继承表达角色差异并各自声明代表性操作，权限校验统一收敛到认证服务；商品与预约分别关联各自的枚举状态，并把「仅允许转换表内的状态变更」作为领域对象自身的规则，而非散落在各服务中。

关联与多重性严格对应一致性需求：卖家 1 对 0..* 商品；商品 1 对 0..* 预约（历史保留，但任一时刻至多一条有效预约，见约束注释）；预约 1 对 0..1 交易确认；收藏与举报均带用户–商品唯一约束。应用服务层的八个服务对应模块编排用例，其中预约服务对商品的依赖（校验状态与版本号）是乐观锁的落点；所有业务服务在关键操作后调用日志服务，为保持图面清晰仅画出代表性连线并在注释中说明。

这样设计的原因：实体与服务分层实现关注点分离——实体封装数据与状态规则，服务封装用例编排，二者与实验三的实现结构（ORM 模型与控制器）天然对应，降低从设计到实现的映射成本；角色继承复用了公共属性与权限抽象，避免三个角色类重复字段。

2.3 结构图（二）：组件图

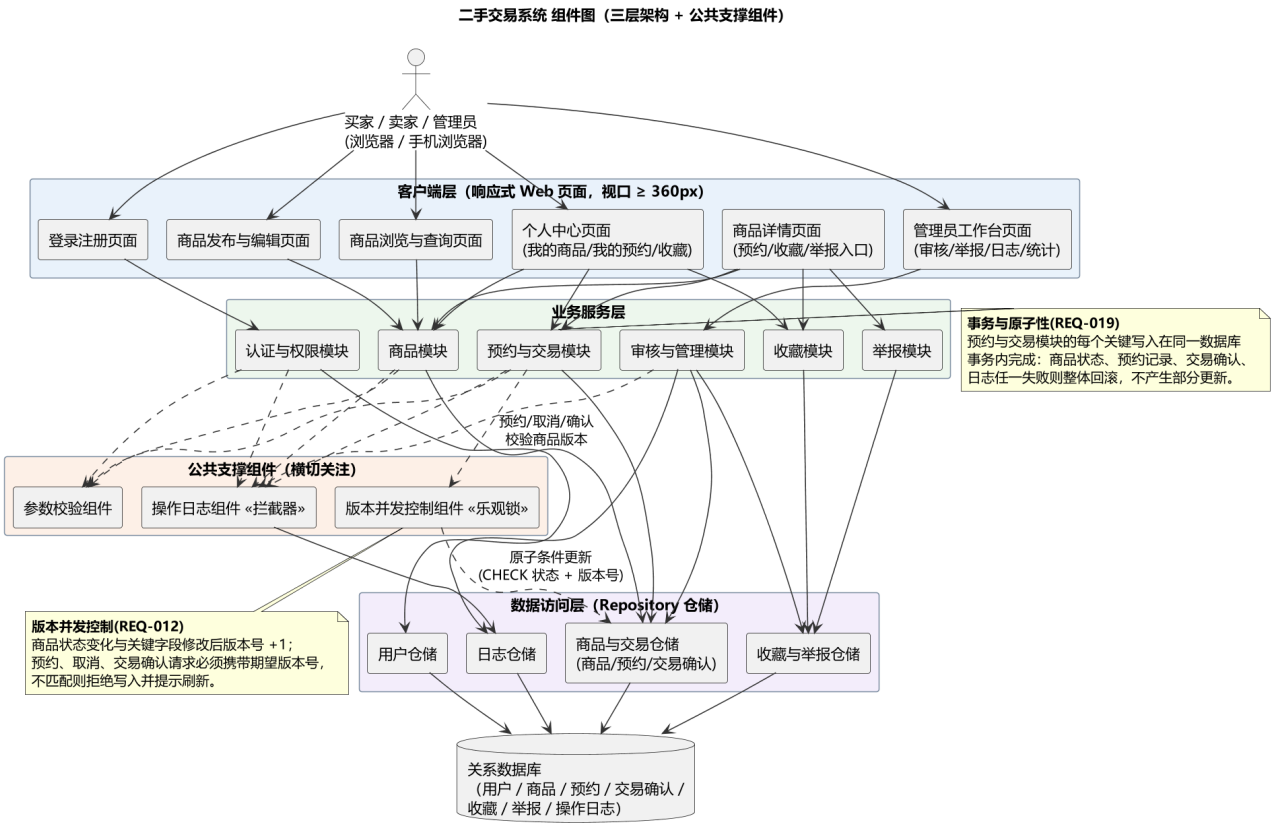


图 2-2 UML 组件图（结构性图）

组件图展示三层架构的物理模块划分：客户端层六个页面组件、业务服务层六个业务模块、数据访问层四个仓储与一个数据库，以及横切的三个公共支撑组件。模块交互关系为：页面调用业务模块，业务模块经仓储访问数据库；审核与管理模块对商品的操作复用商品模块回写状态，而不是绕过商品模块直改数据，保证状态转换始终受 REQ-006 转换表约束；预约与交易模块的全部关键写入经版本并发控制组件执行「状态 + 版本号」原子条件更新，并在同一事务内完成商品状态、预约记录与日志的同步写入（REQ-019）。

设计原因：横切关注（日志、并发控制、参数校验）独立成组件，由各业务模块复用，避免重复实现并保证口径一致；依赖方向单向向下，任何跨层调用都被显式画出，便于检查越权路径；移动端易用性约束（REQ-020）集中在客户端层的响应式页面中解决，不侵入业务层。

2.4 行为图（一）：用例图

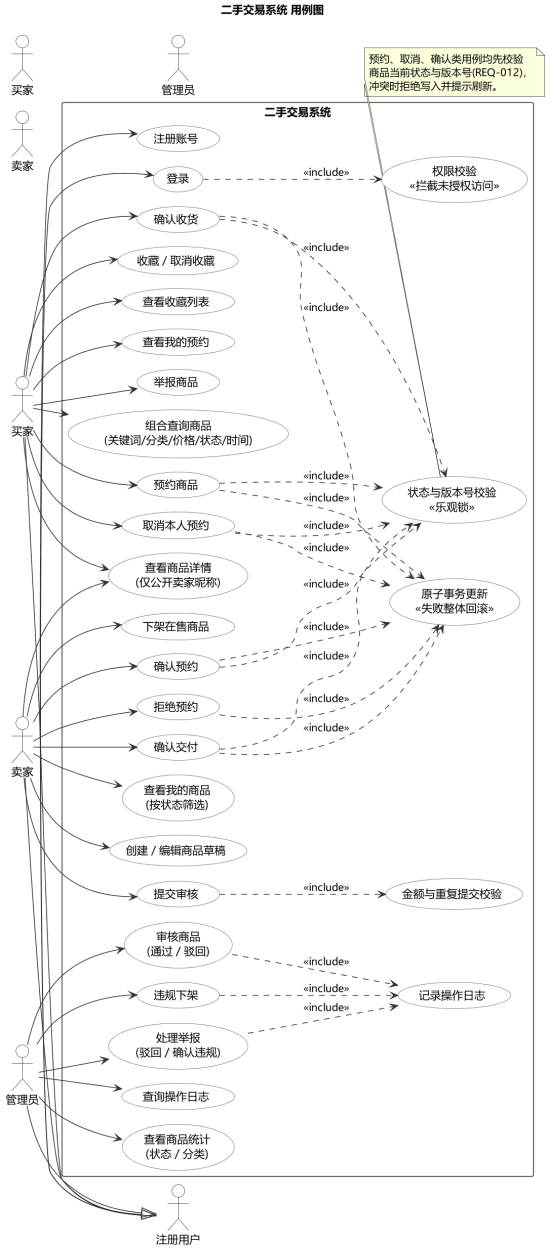


图 2-3 UML 用例图（行为性图）

用例图中买家、卖家、管理员三个参与者泛化自「注册用户」，注册与登录挂在父参与者上，体现三类角色共享账户体系而权限各异。系统边界内的用例覆盖了表 1-1 的全部功能域，实验一的 19 条功能需求均有对应用例。

图中的包含关系把横切规则抽取为被包含用例：「金额与重复提交校验」被提交审核包含；「状态与版本号校验」被预约、取消、确认类用例包含；「原子事务更新」被全部关键状态变更用例包含；「记录操作日志」被审核与处置类用例包含。这些被包含用例与类图中的服务方法、组件图中的横切组件一一对应，三张图可以互相印证，这是设计一致性的直接体现。

2.5 行为图（二）：状态图

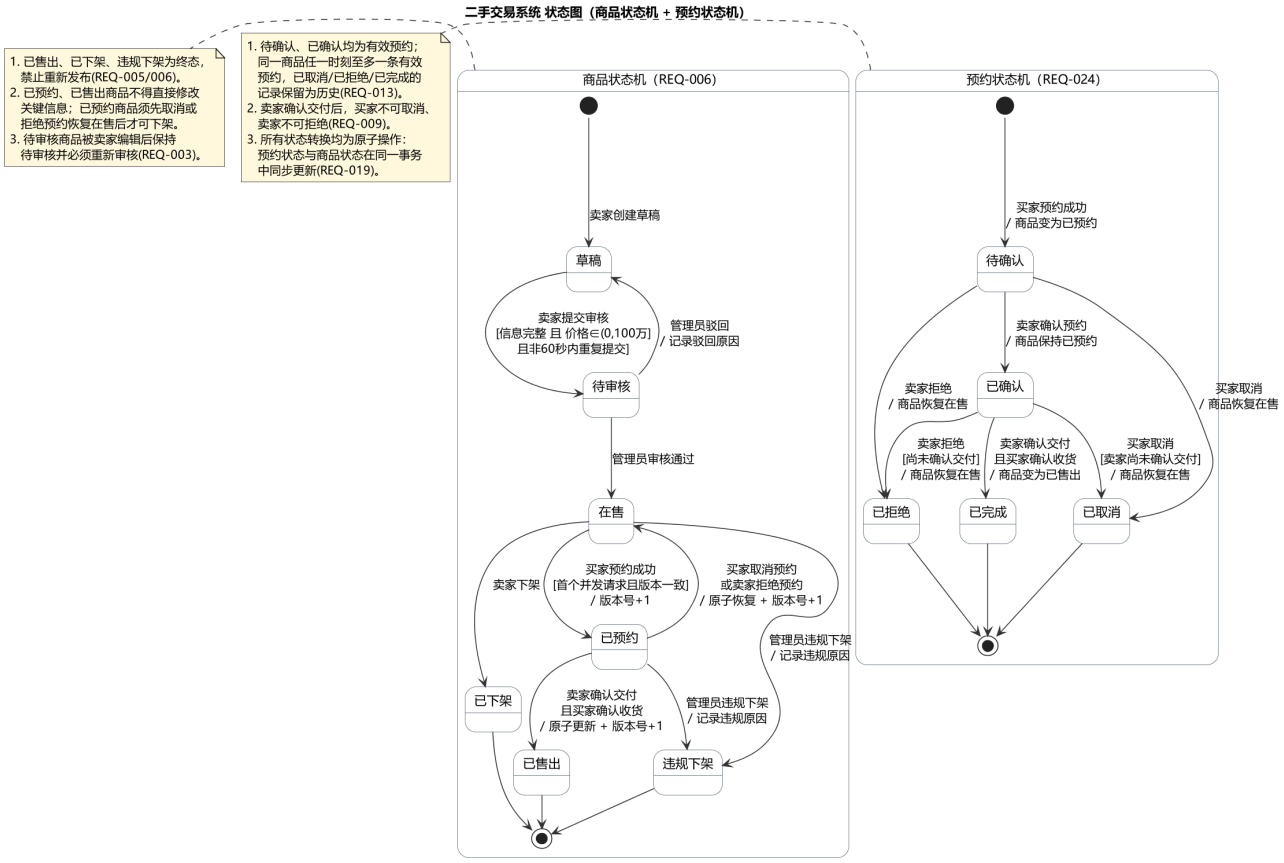


图 2-4 UML 状态图（行为性图）：商品状态机与预约状态机

状态图包含两个状态机。商品状态机（REQ-006）共七种状态，其中已售出、已下架、违规下架为终态：草稿经卖家提交审核进入待审核；待审核经管理员通过变为在售、经驳回退回草稿；在售可因买家预约变为已预约、因卖家下架或管理员违规下架离开；已预约可因买家取消或卖家拒绝恢复在售（原子恢复），仅在卖家确认交付且买家确认收货后变为已售出。每条转移标注了触发角色、守卫条件与版本号递增动作。

预约状态机（REQ-024）共五种状态：待确认与已确认为有效预约，已取消、已拒绝、已完成为终态；取消与拒绝在卖家确认交付之前允许，确认交付之后双方均不可反悔。两个状态机的转换与商品状态严格同步（如预约成功则商品变为已预约），同步更新在单一事务内原子完成。

设计原因：本系统的业务核心就是两个对象的状态流转，把转换约束画成状态机后，实现时可直接落地为转换表（数据或规则代码），测试时可按「每条允许转移正向验证、每条禁止转移反向验证」设计用例，是三类图中对实现与测试指导性最强的一张。

2.6 行为图（三）：活动图

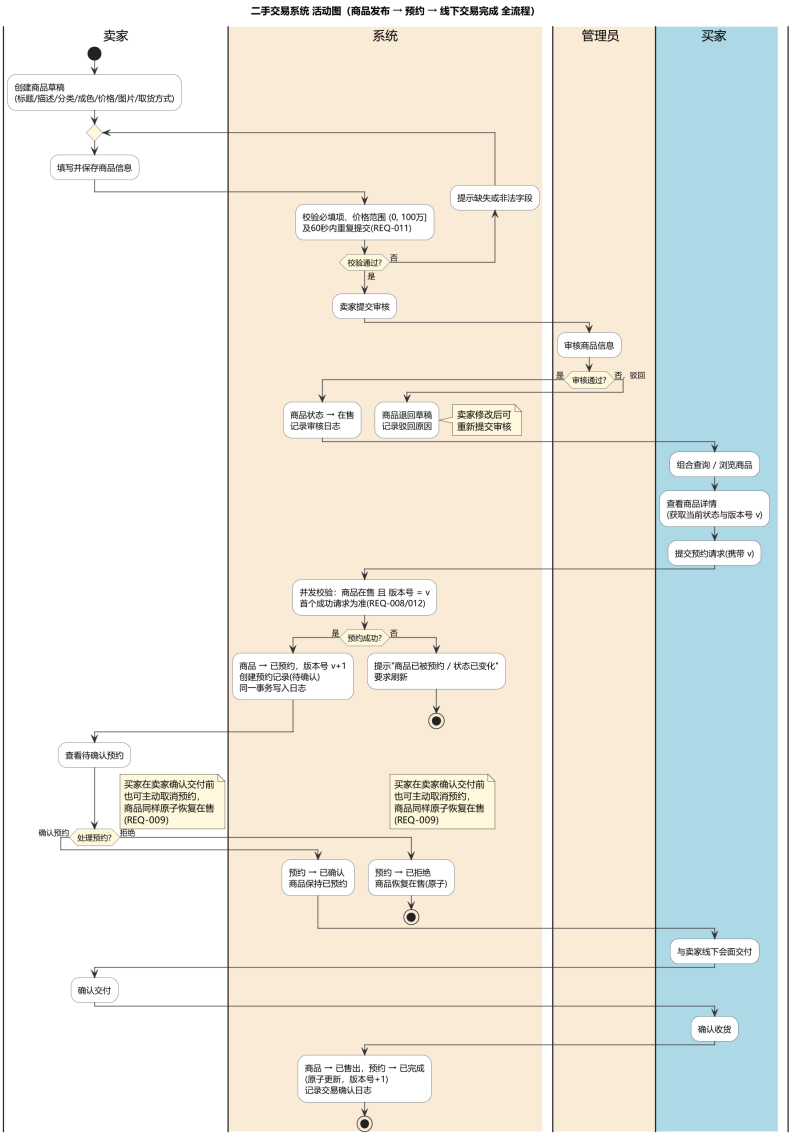


图 2-5 UML 活动图（行为性图）：发布-预约-交易全流程

活动图以卖家、系统、管理员、买家四条泳道刻画主流程：卖家创建草稿并填写信息，系统校验必填项、价格范围与 60 秒重复提交（校验不通过回到填写）；卖家提交审核后管理员审核，驳回则退回草稿重新修改；审核通过商品在售，买家组合查询、查看详情并携带版本号提交预约；系统执行并发校验，首个成功请求创建唯一有效预约并递增版本号，失败请求收到已被预约提示；卖家确认或拒绝预约（拒绝则商品原子恢复在售，流程回到可预约状态；买家亦可在交付确认前取消）；确认后双方线下交易，卖家确认交付、买家确认收货，系统原子地将商品置为已售出、预约置为已完成并记录日志。

设计原因：活动图擅长表达跨角色的协作流程与分支回退，是向甲方与开发队友解释业务全貌的首选图；它与状态图互补——活动图是流程视角，状态图是单个对象的生命周期视角，两者对「驳回回退、取消恢复、并发胜负」三处易错点的表达相互印证。

2.7 交互图：时序图

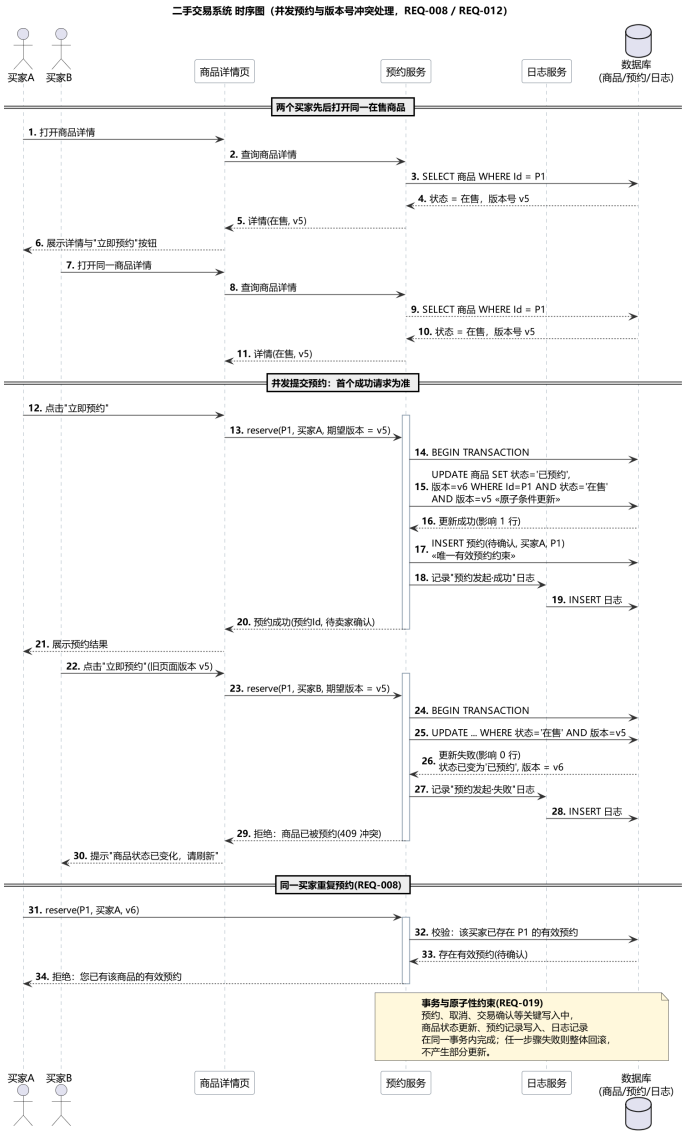


图 2-6 UML 时序图（交互性图）：并发预约与版本号冲突处理

时序图选取本系统技术风险最高的场景：两个买家并发预约同一件在售商品（当前版本 v5）。买家 A 的请求通过原子条件更新（UPDATE ... WHERE 状态=在售 AND 版本=v5）成功，商品转为已预约、版本递增为 v6、插入唯一有效预约并写入日志，全部在同一事务内完成；买家 B 基于旧页面版本 v5 的请求因条件不满足更新零行，系统拒绝写入、返回 409 冲突并提示刷新，同时记录失败日志；末尾的可选片段覆盖同一买家重复预约的拒绝路径。

设计原因：时序图按时间轴展开消息与激活期，能同时表达调用次序、事务边界与失败分支，是并发控制设计评审时最有效的沟通工具；图中事务与原子性注释直接对应可靠性需求 REQ-019，可作为实现阶段编写集成测试的规格。

三、各功能模块 UI 界面设计与功能描述

3.1 设计规范与制作方式

全部界面按移动端优先设计：逻辑宽度 360px（对应 REQ-020 的最低保证视口），原型以 2 倍分辨率渲染导出；每张设计图包含左右两个界面状态（正常态与边界/异常态），异常态直接对应具体需求条款（校验错误、并发冲突弹层、权限锁定、状态同步），确保「界面即需求的可视化」。

视觉规范：主色为暖橙（发布与预约等主动操作），状态标签使用统一色板并与状态图的七种商品状态、五种预约状态一一对应——草稿灰、待审核蓝、在售绿、已预约橙、已售出灰蓝、已下架浅灰、违规下架红；底部导航为首页、收藏、发布、我的四项。原型使用代码化 UI 设计工具（HTML/CSS 设计系统）绘制高保真稿，经浏览器渲染后导出 PNG，其设计系统做法（色板、组件、状态样式复用）与 Figma/Pixso 等工具一致，制作过程与 AI 辅助情况见第四章。

界面	名称	覆盖模块	呈现的状态
界面 1	用户注册与登录	M1	登录失败提示；注册表单校验（手机号唯一、密码规则、角色选择）
界面 2	商品浏览与组合查询	M2	首页商品卡片流；筛选面板与价格区间即时校验
界面 3	商品详情	M2/M3/M4/M5	在售详情（卖家公开昵称）；已预约状态与版本冲突弹层
界面 4	商品发布与我的商品	M2	发布表单校验（金额、必填）；按七种状态筛选与编辑锁定
界面 5	我的预约（买家）	M3/M7	多状态预约列表与差异化操作；交易四步时间线
界面 6	卖家预约处理与交付	M3	待处理预约确认/拒绝；双方确认进度与锁定规则
界面 7	个人中心与收藏	M4/M7	双角色身份与业务清单入口；收藏状态实时同步
界面 8	管理员工作台	M5/M6	审核通过与驳回（原因必填）；举报处置、统计、日志查询

表 3-1 八组界面的模块覆盖与状态设计

3.2 界面 1：用户注册与登录（模块 M1）



图 3-1 用户注册与登录（模块 M1）

登录页提供手机号与密码输入，登录失败时给出明确原因并提示已记录日志（REQ-014 同时记录失败操作）；注册页包含手机号（唯一性即时校验）、用户名（提示可修改且允许重复）、密码规则校验与角色选择（买家/卖家，管理员账号由平台预置不开放注册），底部说明密码仅以不可逆摘要保存（REQ-018）。

3.3 界面 2：商品浏览与组合查询（模块 M2）



图 3-2 商品浏览与组合查询（模块 M2）

首页为搜索栏、分类快捷标签与双列商品卡片流，卡片展示缩略图、标题、价格、成色、卖家昵称与状态标签，普通用户列表自动过滤已下架与违规下架商品（REQ-007）。筛选面板支持分类、价格区间、商品状态与发布时间的任意组合，价格区间下限大于上限时即时校验报错，并预估匹配数量、空结果有明确提示。

3.4 界面 3：商品详情（模块 M2/M3/M4/M5）

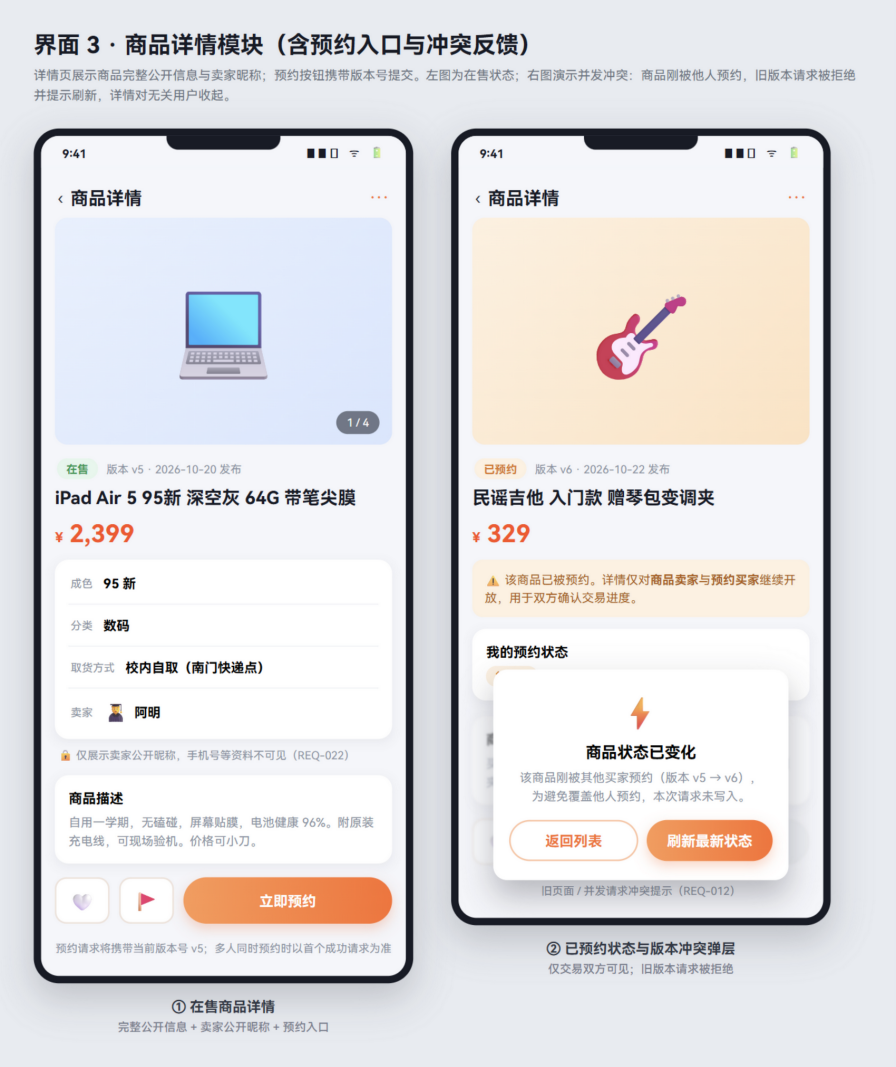


图 3-3 商品详情（模块 M2/M3/M4/M5）

左屏为在售商品详情：完整公开信息、成色/分类/取货方式、卖家公开昵称（明确提示不展示手机号，REQ-022），底部为收藏、举报与立即预约入口，按钮下方标注预约将携带当前版本号（REQ-012）。右屏为已预约状态：详情仅对交易双方继续开放，无关用户看到的是并发冲突弹层——商品刚被他人预约（版本 v5 变为 v6），本次请求未写入，提供刷新最新状态操作，直观演示乐观锁的失败路径。

3.5 界面 4：商品发布与我的商品（模块 M2）



图 3-4 商品发布与我的商品（模块 M2）

发布表单覆盖七项必填信息与图片上传，价格输入含金额范围即时校验（0 至 100 万元，REQ-011），操作为保存草稿与提交审核双按钮。我的商品页按全部/草稿/待审核/在售/已预约/已售出/已下架筛选，不同状态提供差异化操作：待审核提示编辑后须重新审核（REQ-003），已预约商品锁定编辑并提示须先处理预约方可下架（REQ-005）。

3.6 界面 5：我的预约与交易确认（模块 M3/M7）



图 3-5 我的预约与交易确认（模块 M3/M7）

左屏为按状态筛选的预约列表：待确认状态提供取消预约按钮；卖家已确认交付的预约高亮确认收货按钮；已完成交易灰化展示；全部操作受前置条件约束（REQ-009/010）。右屏为预约详情的交易四步时间线（提交预约、卖家确认、卖家确认交付、买家确认收货），已完成的步骤标注时间与版本号变化，收货按钮下方说明确认操作原子完成、失败整体回滚（REQ-019）。

3.7 界面 6：卖家预约处理与交付确认（模块 M3）



图 3-6 卖家预约处理与交付确认（模块 M3）

左屏为收到的待确认预约，提供确认预约与拒绝预约双操作，页面提示确认交付前买家可取消、确认交付后不可再拒绝或取消（REQ-009/024）。右屏为交付确认页：双方确认进度卡展示卖家已确认交付与买家待确认收货，附最近状态流转记录（含每次状态变化与版本号递增、日志落库说明，REQ-014），帮助双方对交易进度形成一致预期。

3.8 界面 7：个人中心与收藏（模块 M4/M7）

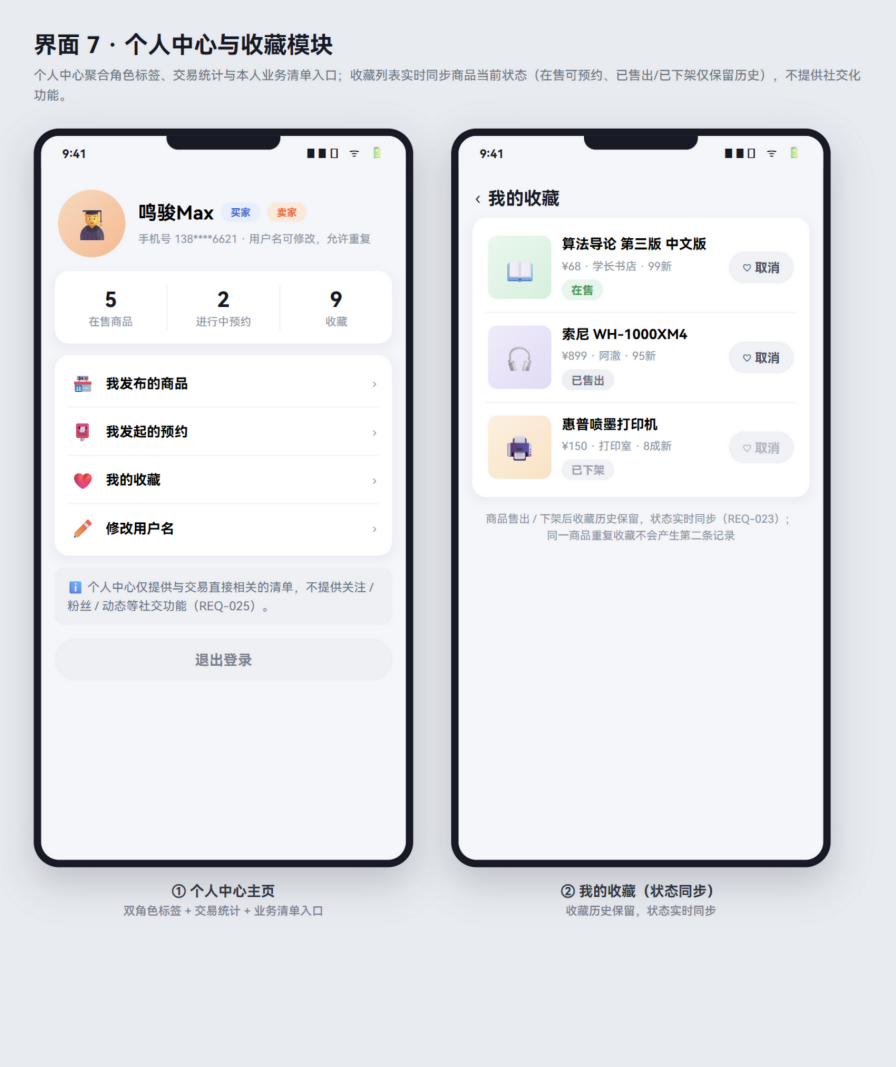


图 3-7 个人中心与收藏（模块 M4/M7）

个人中心展示双角色标签（同一账号可同时是买家与卖家）、交易统计与我的商品/我的预约/我的收藏入口，明确说明不提供关注、粉丝与动态功能（REQ-025）。收藏列表实时同步商品当前状态：在售商品可继续预约，已售出与已下架商品保留历史记录并灰化展示（REQ-023），同一商品重复收藏不会产生第二条记录。

3.9 界面 8：管理员工作台（模块 M5/M6）



图 3-8 管理员工作台（模块 M5/M6）

左屏为审核工作台：待审核商品卡片展示校验预检结果（图片、描述、价格、重复提交检查），提供通过与驳回操作，驳回原因必填（驳回后退回草稿，REQ-004）。右屏为运营页：待处理举报的单层处置（驳回举报或确认违规并下架，REQ-026）、按状态与分类的商品统计（空分组计零，REQ-016）、以及按操作者/对象/类型/时间四条件的日志检索，成功与失败操作均可见（REQ-015）。

四、大模型辅助工具的使用记录与分析

本章如实记录实验过程中大模型（GLM，经 ZCode 编码代理调用）的每一次主要使用，按「使用场景 – 生成结果 – 优缺点分析 – 修改优化后的结果 – 我的思考」组织，包括失败与返工的真实过程；最后一节是整体的优缺点分析与工作流思考。

4.1 场景1：UML 图选型与 PlantUML 语法样例获取

使用场景：实验开始时需要确定绘制哪几种 UML 图，并快速掌握 PlantUML 的基本写法。

生成结果：模型给出了三类图的划分清单与选型建议，并提供了类图、用例图的 PlantUML 样例代码（一个通用电商示例：User、Order、Product 等）。

优点：语法入门非常快，样例可以直接渲染跑通，省去了通读文档的时间；三类图的适用场景讲解帮助我快速完成选型。

不足：样例内容与本项目完全无关（没有状态机、审核流与并发控制），如果直接采用会造成图与需求脱节；样例也未覆盖本项目需要的组合状态机、泳道、自动编号等写法。

修改优化后的结果：只保留了语法骨架，参与者、类、状态、守卫条件全部依据实验一的 26 条需求逐条重写；样例中没有的写法（组合状态机、四泳道活动图、autonumber 时序图）另行查阅语法资料补充。

我的思考：这一步让我意识到「问什么」比「问出什么」更重要：样例的价值只在于语法，如果把通用示例当成设计本身，图就成了无根之木。我采取的做法是先把实验一需求逐条编号过一遍，再让模型按编号产出，让它「填空」而不是「自由发挥」。

4.2 场景2：类图生成与布局迭代（两次）

使用场景：依据需求文档让模型生成类图初稿，作为领域模型设计的起点。

生成结果：初版（v1）按「用户与权限、商品域、交易域、辅助业务、平台管理」五个业务包组织，约 20 个类，属性与多重性标注完整。

优点：类的枚举很完整，覆盖了七张表的实体与八个服务，一次性给出约束注释（如唯一有效预约、乐观锁版本号），作为检查清单很有价值。

不足：模型不感知渲染效果：五个包之间的跨包依赖（认证服务指向用户、预约服务指向商品等）导致渲染后连线大量交叉，可读性差；服务与实体混排，分层职责在图面上不清晰。这类视觉质量问题是渲染后人工检查才发现的。

修改优化后的结果：重组为「实体层 + 应用服务层」两个包（v2）：服务只与自己的主实体相连，跨层依赖仅保留预约服务到商品（版本校验）等三条关键线；日志横切关系改用注释表达并说明省略原因。重新渲染后交叉明显减少，分层一目了然。

我的思考：v1 到 v2 的返工教会我：类图的「正确」有两个层面——语义正确（类和关系对）与表达正确（读者看得清）。模型保证了前者，后者只能靠渲染出来人眼验收。这也让我重新审视了包的粒度：包划分得越细，跨包依赖越多，图越乱；按层组织反而更适合自动布局工具。

4.3 场景3：状态图绘制与语法限制踩坑

使用场景：绘制商品状态机与预约状态机，把 REQ-006 与 REQ-024 的转换规则图形化。

生成结果：模型给出了两个组合状态机的完整代码：七种商品状态、五种预约状态、每条转移带触发者与守卫条件，终态有明确标注。

优点：状态与转移和需求逐条对得上，守卫条件（如双方确认后才能售出）与动作（版本号递增）表达完整，基本可直接使用。

不足：初稿在两个组合状态机之间画了一条依赖连线，PlantUML 渲染直接报错（组合状态之间不允许该关系），模型事先没有提示这一语法边界，产生了一次返工。

修改优化后的结果：删除该连线，把「预约成功时创建待确认预约」的关联改写在注释中说明；重新渲染通过，两个状态机并排呈现，注释集中解释终态、唯一有效预约与原子性三条关键规则。

我的思考：报错反而是收获：为了改对，我去查了 PlantUML 状态图的语法文档，顺带弄清了组合状态与子状态机的语义边界。这让我确认了一个判断——AI 出错的地方，往往正是我自己知识的地基不牢的地方；修错的过程比一次成功学到得多。

4.4 场景4：时序图细化

使用场景：为并发预约这一关键场景绘制时序图，表达乐观锁与事务边界。

生成结果：模型给出了基本消息序列：两个买家、预约服务与数据库四个参与者，先后打开详情、提交预约的主干流程。

优点：交互次序正确，参与者划分合理，作为骨架是合格的。

不足：缺少事务边界的表达（BEGIN/COMMIT 没有体现）、没有覆盖同一买家重复预约的分支、失败响应的语义不明，直接使用会掩盖最重要的失败路径。

修改优化后的结果：补充了三段式分隔、自动编号、日志服务参与者、原子条件更新的 SQL 式说明（UPDATE ... WHERE 状态=在售 AND 版本=v5）、409 冲突响应与刷新提示、重复预约的可选片段，以及贯穿参与者的事务原子性注释（REQ-019）。

我的思考：模型的初稿只写了「成功路径」，而系统设计的关键恰恰在失败路径。我由此给自己立了一条规矩：凡是涉及并发与状态变更的图，必须自己先列出所有失败分支，再检查 AI 画没画——「让失败路径可见」是评审这类图的第一检查项。

4.5 场景5：UI 原型设计与视觉修复

使用场景：为八个模块共十六屏设计高保真 UI 原型并导出图片。

生成结果：基于统一设计系统（色板、组件样式、状态标签色）生成的原型代码，浏览器渲染后截图导出，每屏两倍分辨率、宽 360 逻辑像素。

优点：像素级可控、样式天然统一、修改即所见；界面状态由需求驱动（每屏标注对应需求编号），十六屏一次成型，效率远高于在图形工具里逐个拖拽。

不足：首版存在视觉问题：左右两台手机样机高度不一致（内容高度不同导致），整体不齐——这再次说明模型无法自评视觉质量；此外个别屏信息密度不均，需要逐屏检查。

修改优化后的结果：为样机容器设置统一最小高度后重新渲染，并对全部八张图逐屏目检：对照需求核对冲突弹层、按钮锁定、状态同步等关键细节后定稿。

我的思考：UI 是「AI 最强也最危险」的环节：产出快到让人容易跳过检查直接采用。我强迫自己按「每屏对应哪些 REQ、异常态是否呈现」逐屏打钩，才发现高度不齐的问题。设计评审清单比设计速度更决定最终质量。

4.6 场景6：实验报告撰写辅助

使用场景：按课程提交要求组织报告结构并起草各章节文字。

生成结果：模型根据本报告大纲（项目说明、架构与 UML、UI 设计、工具使用记录、总结）生成了章节骨架与初稿。

优点：结构完整性好，能对照提交要求逐项自检不遗漏；长内容自动建议表格化表达，可读性更好。

不足：行文容易冗长、口语化；个别表述倾向于把过程「美化」（例如把渲染失败、布局返工略去不提），需要人工核对真实性后保留失败记录。

修改优化后的结果：逐节核对与需求编号的一致性，删减重复表述；明确保留第四章中失败与返工的真实过程，使工具使用记录可信。

我的思考：「美化倾向」是我在文字类输出里最警惕的问题：AI 会本能地让过程显得顺利。而实验报告的价值恰恰在于如实记录。我的对策是把每次返工当场记成一行流水（改了什么、为什么改），写报告时照单誊录，不靠事后回忆——这也顺带成了天然的真实性证据。

4.7 优缺点综合分析

综合六次使用，大模型在本实验中的价值与局限可归纳如下。

维度	优点	不足
效率	模板与语法（PlantUML、设计系统）生成快，把重复劳动压缩为修改；长流程多屏产出一次成型	产出后仍需人工逐项核对，校对成本随产出量上升
完备性	从需求到设计元素的映射覆盖好，能列出应画的类、状态与用例，辅助查漏	不了解项目上下文时产出通用内容，与本系统状态机、审核流等特征脱节
一致性	多张图之间（类图方法、用例包含关系、组件横切）容易保持相互印证	对工具语法边界把握不足（组合状态连线报错），可能产生返工
质量自评	结构化文本（表格、清单）质量稳定	无法自评视觉质量（连线交叉、界面高度不齐），必须渲染后人工目检
真实性	按给定事实陈述时准确	存在「美化」倾向，可能略去失败过程或编造细节，需要人工把关

表 4-1 大模型辅助软件设计的优缺点汇总

4.8 总结：大模型如何辅助软件设计，以及「人只做检查校对」的工作流

从本次实验的体验出发，大模型对软件设计的辅助可以归纳为四种方式：一是映射加速——把编号化的需求批量映射为类、状态、用例、界面等设计元素；二是初稿生成——PlantUML 代码、UI 原型、报告文字都能以草稿形式快速产出；三是完备性检查——用「需求编号到设计元素」的覆盖矩阵发现遗漏；四是多图一致性校验——类图、组件图、用例图、时序图之间的概念对应关系可由模型逐条核对。而人真正不可替代的是业务决策（范围取舍、终态定义、权限边界、验收标准）与视觉/语义的双重把关。

在此基础上，可以设计一套让「人主要完成检查校对」的工作流，把人工从生产环节移到验收环节：

- 第一步（人定标准）：输入规约——需求编号化并附验收标准（本实验已完成：REQ-001 至 REQ-026），这是机器可执行检查的前提；
- 第二步（AI 产出清单）：生成「设计元素清单」——类/状态/用例/组件/界面与需求编号的映射表，提交人工评审点一：每条功能需求至少被一个设计元素覆盖、无编造；
- 第三步（AI 产出制品）：按清单生成 puml 源码与界面原型，配自动化校验——PlantUML 渲染作为语法门禁（渲染失败即打回），再用脚本生成「图-需求覆盖矩阵」验证没有需求掉队；提交人工评审点二：查看渲染结果，判断布局可读性与业务语义正确性；
- 第四步（AI 产出文档）：起草报告与说明文字，提交人工评审点三：真实性（保留失败记录）与表达质量；
- 第五步（人保留决策权）：范围、状态机终态、权限模型、验收标准等关键设计决策始终由人确认后冻结。

我的结论是：这一工作流在当前工具条件下基本可行——本实验实际上就是按上述一二三步执行的，人工时间主要花在三个评审点上；但「人只做检查」有两条底线不能省：其一，模型不能自评视觉质量，渲染结果必须人看；其二，模型的产出会失真或美化，验收标准必须尽量前移为机器可执行的检查（渲染门禁、覆盖矩阵），使人的校对有据可依，而不是凭印象放行。

五、实验总结

本实验把实验一的 26 条需求翻译为互相印证的六张 UML 图与八组界面设计，最大的收获有三点。第一，图要按项目特征选型：本系统的核心是两个状态机的受控流转与一处高并发场景，因此状态图与时序图承担了最关键的表达任务，而类图与组件图保证了设计与后续实现的映射关系。第二，设计要为实施服务：状态转换表、版本号乐观锁、唯一有效预约约束、仓储分层等设计都落到了可实现、可测试的细节（数据表、条件更新语句、拦截器），而不是停留在概念层面。第三，对大模型辅助设计的边界有了切身体会：它显著加速了初稿生产与完备性检查，但视觉质量自评与业务决策仍必须由人完成。

后续实验三将按组件图的分层结构实现原型，优先完成预约与交易模块（技术风险最高），并以状态图的转换表作为测试设计的直接依据。

附录：提交物清单

文件	说明
实验报告.pdf	本报告
UML/UML类图.puml、UML类图.png	结构性图（必画）：实体层与应用服务层
UML/UML组件图.puml、UML组件图.png	结构性图：三层架构与公共支撑组件

文件	说明
UML/UML用例图.puml、UML用例图.png	行为性图：三类角色与系统边界
UML/UML状态图.puml、UML状态图.png	行为性图：商品与预约两个状态机
UML/UML活动图.puml、UML活动图.png	行为性图：四泳道全流程
UML/UML时序图.puml、UML时序图.png	交互性图：并发预约与版本冲突
UI界面/界面1.png 至 界面8.png	八个功能模块共十六屏高保真界面设计

表 A-1 提交物清单